

CIdOps Cheatsheet

Yanick Egli & Doriano Di Pierro & Fabio Zahner

1 General Definitions

NIST Cloud: On-demand self-service, Broad network access, Resource pooling (multi-tenant), Rapid elasticity, Measured service (pay-as-you-go).

Cloud Operations: manage/optimize cloud services & infra.

GitOps: Git = single source of truth; declarative, versioned CI/CD.

DevOps: dev+ops; automation, CI/CD, monitoring, agile.

CI: developers integrate into the shared mainline frequently (small commits, short-lived branches); every push auto-triggers build + tests (unit/integration), catching conflicts/breakage early and keeping the trunk green & releasable. Avoids big-bang "integration hell"; fast feedback, fail fast.

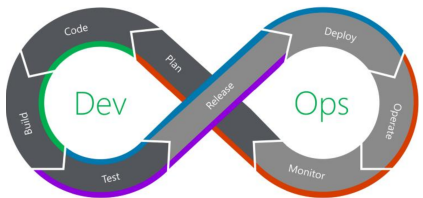
CD: once CI passes, the change is auto-packaged & promoted through environments (dev→staging→prod). **Continuous Delivery** = automated up to a manual gate before prod (one-click release; artifact always deployable); **Continuous Deployment** = no gate, every green change auto-released to prod.

Platform Engineering: internal self-service dev platforms (IDP); faster shipping.

How they relate: DevOps (culture & practice) → CI/CD (automates it via pipelines) → Platform Engineering (productizes it as a self-service IDP to scale across teams).

DevOps 6 pillars: Cultural mindset, Automation-first, Continuous Processes, Monitoring/Feedback, Rapid+Reliable Releases, Security.

1.1 DevOps Cycle



Arcs → stages: **CI:** Code+Build+Test, **CDeployment:** Release+Deploy (+Test), **CDelivery:** Test+Release (manual prod), **CFeedback:** Operate+Monitor+Plan.

Stages (activities | tools): Plan (requirements, roadmap, arch sketch, risk/rollback | Jira, Asana, GitHub Issues, Trello), Code (write/refactor, local build/test, peer review | GitLab, GitHub, Bitbucket), Build (compile, container images, publish artifacts, versioned packages

| GitLab CI, GH Actions, Jenkins, Maven/Gradle, Docker/Podman, Artifactory, Quay), Test (unit/integration/E2E, perf/load, sec scans, coverage, quality gates | JUnit/pytest, Selenium/Cypress-Playwright, JMeter/k6, Snyk/Trivy, SonarQube), Release (promote artifact, immutable version, release notes, signing | Cosign, GnuPG, GitLab/GH Packages), Deploy (pull artifact, apply manifest, strategy, post-deploy validation, rollback | ArgoCD, Flux, Helm, Kustomize, Terraform/OpenTofu, Ansible, Flagger), Operate (troubleshoot, scale, patch, secret rotation, drift detect, backups, compliance | PagerDuty, Vault, Velero, HPA/VPA, Falco), Monitor (SLI/SLO, dashboards, alerts, synthetic checks, correlate logs/metrics/traces | Prometheus, Grafana, Loki, Jaeger, ELK, Alertmanager).

CD Deployment Strategies:

- **Rolling:** gradual update, minimal downtime
- **Blue-Green:** two envs, fast switch
- **Canary:** small user group first
- **Feature Flag:** deploy, toggle later
- **Dark Launching:** real traffic, hidden from users

1.2 Platform Engineering

Core Responsibilities: enable self-service, standardize runtimes/tooling, abstract infra, enforce compliance & security, provide observability.

8 Building Blocks (top→bottom): Self-Service Portal (IDP), Cost & Capacity, Observability, Policy & Security, Package & Artifact Mgmt, CI/CD Pipelines & Deployment, Infrastructure as Code, Compute & Orchestration.

Top Container Challenges (CNCF 2025): Cultural changes (47%), Lack of training (36%), Security (36%), CI/CD (35%), Monitoring (35%), Complexity (34%), Scaling, Networking, Logging, Service Mesh.

2 GitLab Pipelines

```
default: { image: golang:1.24-alpine } # fallback image for all jobs
stages: [build, test, release, deploy, post-deploy]

build: # DevOps cycle -> Build
  stage: build
  script:
    - go mod download
    - CGO_ENABLED=0 go build -o bin/app ./...
  artifacts: { paths: [bin/app] } # persist for later stages

lint-test: # DevOps cycle -> Test
  stage: test
  image: golangci/golangci-lint:v1.64.5-alpine
  script: [golangci-lint run -v]

build-docker: # DevOps cycle -> Release (publish image)
  stage: release
  image: docker:28
  services: [{ name: docker:28-dind, alias: docker }] # dind
```

```
variables: { IMG: $CI_REGISTRY_IMAGE: $CI_COMMIT_SHORT_SHA }
before_script:
  - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
script:
  - docker build -t $IMG -f Dockerfile.prod .
  - docker push $IMG

deploy-prod: # DevOps cycle -> Deploy
  stage: deploy
  image: alpine/kubectl:1.35
  script:
    - yq -i ".spec.template.spec.containers[0].image=\"$IMG\"" k8s/deployment.yaml
    - kubectl apply -f k8s/deployment.yaml
  environment: { name: production, url: https://demo.example.com }
  rules: [{ if: '$CI_COMMIT_BRANCH == "main"' }] # prod only on main

post-deploy: # DevOps cycle -> Operate/Monitor
  stage: post-deploy
  script: [echo "smoke test + health check"]
```

2.1 Core Concepts & DSL Mapping

Pipeline = automated steps → faster, safer, reliable. Traceable logs, parallel jobs, consistent envs kill "works on my machine."

5-stage flow: commit/MR/tag → build (compile, image, package) → test (unit/integration/security/perf) → release (publish to registry) → deploy (target env) → post-deploy (smoke, health, rollback).

Map to DevOps cycle: the pipeline automates the loop's middle - build→Build, test→Test, release→Release, deploy→Deploy; triggered by Code (commit/MR/tag), feeds Operate/Monitor. **CI** = build+test; **CD** = release+deploy.

2.2 Jobs, Stages & Rules

.gitlab-ci.yml YAML DSL. **Job** = smallest unit, belongs to one **stage**, runs script on a runner. Same stage → parallel; stages → sequential.

rules: when a job runs (modern replacement for only/except). **needs:** builds DAG, start before stage finishes. **dependencies:** which jobs' artifacts to pull (no ordering). **workflow:rules:** gate the whole pipeline.

```
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never
    - if: $CI_COMMIT_BRANCH

deploy:
  stage: deploy
  needs: [build] # DAG, skip stage wait
  dependencies: [build] # pull build's artifacts
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
    - when: never
```

2.3 Cache, Artifacts & Services

Cache: shared across jobs/pipelines, speed up repeat work (deps, compiled objects).

Artifacts: files persisted to GitLab for downstream stages or download; declare via dependencies/needs.

Services: sidecar containers (DB, dind, API) reachable via alias.

```
integration-test:
  services: [postgres:17, alias: db]
  variables: { POSTGRES_DB: app, POSTGRES_USER: u, POSTGRES_PASSWORD: p }
  script: [psql -h db -U $POSTGRES_USER -d $POSTGRES_DB -c "SELECT 1"]
  cache: { key: deps, paths: [.cache/]}
  artifacts: { paths: [bin/], expire_in: 1h }
```

2.4 Environments

Where code is deployed. **Tiers:** Local (workstation) → Dev (sandbox) → Integration (CI target) → Test/QC (interface tests) → Staging (prod mirror) → Production (live). Linkable to K8s cluster (set up via UI). environment.on_stop unstages; action: stop stops it.

2.5 Push vs. Pull Deployment

Push (Jenkins/GitLab calls cluster): **pro:** easy, flexible targets (VMs/containers, not just K8s); **con:** firewall open, admin access out, pipeline per env.

Pull (agent in cluster, e.g. Argo/Flux): **pro:** secure (no inbound), auto-reconcile, scales identically; **con:** agent per cluster, polling latency.

2.6 GitLab Runner

Worker executing jobs defined in .gitlab-ci.yml. **Scope:** Instance (shared, GitLab-managed) / Group / Project (dedicated). **Executors:** shell, ssh, docker, docker+machine, kubernetes, virtualbox, parallels. Register: gitlab-runner register with URL + token + tags + executor. Jobs pick runner by tags:.

2.7 More Keywords & Predefined Vars

default: fallback image/before_script. before_script/after_script wrap every script. rules:changes: run only if files changed.

Predefined vars: \$CI_REGISTRY[IMAGE], \$CI_REGISTRY_USER/ PASSWORD, \$CI_COMMIT_BRANCH, \$CI_COMMIT_SHORT_SHA, \$CI_PROJECT_DIR, \$CI_JOB_TOKEN (auth API/registry), \$CI_MERGE_REQUEST_IID. Pin images by digest (img@sha256:..), not :latest → flaky.

3 Terraform

IaC: manage & provision infrastructure (compute, storage, network) through code instead of manual processes (Red Hat).

Motivation: replace slow, drift-prone manual console clicking with code; Automation pays off but carries its own upkeep cost.

Benefits: idempotent (declarative, re-apply = same result), drift detection

(state vs. reality), version-controlled (history/review/rollback), consistency (change a source → all references update), reusable & standardized (modules: write once, deploy many, enforce best practices), team collaboration (shared, locked remote state), flexible (variables, no hard-coded secrets).

Declarative vs. Imperative: Terraform is declarative – you describe the desired end state and TF figures out the steps to get there. Imperative (shell scripts, manual CLI calls) instead describes how to reach a result step by step.

Toolchain (4 components): language (HCL – declare infra & vars), state (.tfstate, maps config to real resources), provisioner = Terraform-core plugins: Providers (access a cloud's API: AWS/Azure/GCP/K8s) + Provisioners (remote-exec/local-exec scripts on a resource), backend (where state lives: local or remote S3+lock; init sets it up + downloads plugins). TF doesn't speak directly with an SDK, but rather Terraform -> Provider -> Client SDK. Different providers enable different platforms (AWS, Azure, Kubernetes, ...). **Resources** (resource) are the "what"TF creates and manages (e.g. an EC2 instance). **Data Sources** (data) are read-only lookups of existing infrastructure used as input (e.g. an existing AMI or VPC). A sample in HCL (Hashicorp Configuration Language):

```
terraform {
  required_version = ">= 1.0.0"
  required_providers { aws = { source = "hashicorp/aws", version = ">= 5.0" } }
}

provider "aws" { region = "us-east-1" }
variable "instance_type" { default = "t2.micro" }

resource "aws_instance" "web" {
  ami = data.aws_ami.ubuntu.id
  instance_type = var.instance_type
}

output "public_ip" { value = aws_instance.web.public_ip }
```

To deploy infrastructure, write HCL in files like main.tf, then run terraform init, terraform plan to show changes that would be made, terraform apply to actually apply the changes. Use terraform destroy to delete all made changes.

3.1 OpenTofu

Fork of Terraform v1.5.7 under **Linux Foundation**, MPL-2.0 (TF switched to BUSL). Drop-in replacement, adds **state encryption**. Avoids single-vendor lock-in.

3.2 HCL Features

count (numeric loop, count.index) and **for_each** (map/set loop, each.key/value) for multiple instances. **Data source** example:

```
data "aws_ami" "ubuntu" {
```

```
most_recent = true
filter { name = "name" values = ["ubuntu/
images/hvm-ssd/ubuntu-*"] }
filter { name = "virtualization-type" values =
["hvm"] }
owners = ["099720109477"] # Canonical
}
```

3.3 Variables

Precedence: CLI -var/-var-file > env TF_VAR_<name> > *.tfvars > default. No default & not supplied -> CLI prompts operator. TF_VAR_ prefix ideal for secrets (TF_VAR_access_key=...). depends_on forces explicit ordering; implicit deps come from attribute references.

3.4 State

terraform.tfstate (JSON) maps HCL to real resource IDs (e.g. ARN). **3-way diff** on apply: **NEW** (HCL) vs **PREVIOUS** (.tfstate) vs **EXISTING** (cloud refresh) -> decides actions, detects drift. **Remote backend** (S3) shares state in teams; **state locking** via use_lockfile (S3-native, replaces DynamoDB):

```
terraform {
  backend "s3" {
    bucket = "my-tfstate"
    key = "prod/terraform.tfstate"
    region = "us-east-1"
    use_lockfile = true
  }
}
```

3.5 File & Folder Layout

- modules/
 - shared modules, reusable across envs
- dev/
 - terraform.tf
 - required_version + providers
 - backend.tf
 - remote state (kept out of VCS)
 - providers.tf
 - provider auth/config
 - main.tf
 - resources & data sources
 - variables.tf
 - inputs (alphabetical)
 - outputs.tf
 - returns (alphabetical)
 - locals.tf
 - derived values
 - override.tf
 - loaded last (use sparingly)
- prod/
 - same layout, its own state

Per-env folders → separate state, so a corruption hits only one env.

3.6 Modules

Reusable folder of .tf files (root config is itself a module):

```
module "vpc" {
  source = "../modules/vpc" # or registry/git URL
  cidr = "10.0.0.0/16" # input variable
} # reference an output: module.vpc.vpc_id
```

3.7 Patterns & Recovery

import adopts existing infra; terraform apply -replace=<addr> (or legacy taint) forces a destroy+recreate of a single resource. Region-aware AMI via map:

```
import { to = aws_instance.web id = "i-0abc123"
} # then: apply

variable "AMIS" { type = map(string) default =
  { us-east-1 = "ami-05b1..." } }

ami = var.AMIS[var.region]
```

4 Ansible

Configures provisioned machines (install/configure software, deploy code); *after* provisioning. Idempotent (no-op if already in desired state), agentless, push-based, no statefile/locking. Needs Python on host & targets. Other CM tools: **Puppet**, **Chef**, **CFEngine**, **Juju**.

TF vs Ansible: TF = provisioning, declarative, state, idempotent. Ansible = CM, *hybrid* (imperative tasks, declarative modules), no lifecycle, idempotency *optional*. Both: templating, agentless, OSS. Often combined.

Execution: over SSH, copies generated Python module to ~/.ansible/tmp on target, runs with remote Python, returns JSON on stdout, deletes. Nothing stays installed. **Exceptions:** raw module runs cmd via SSH (no Python needed, e.g. bootstrap); connection: local runs on controller (network devices); Windows via **WinRM**.

4.1 Initial Setup & Inventory

ssh-keygen -t ed25519 -C *@y" on controller, ssh-copy-id user@server, test with ansible all -m ping. **Inventory** (INI or YAML) lists managed hosts grouped, referenced by play's hosts:. ansible-inventory -i inv --list verifies it.

```
[web]
web1.example.com
web2.example.com ansible_host=10.0.0.5
[web:vars]
ansible_user=deploy
```

4.2 Playbooks, Plays & Tasks

Hierarchy: a **playbook** is one YAML file holding 1..n **plays**; a play maps a group of **hosts** to an ordered list of **tasks**; each task is a single **module** call (e.g. **apt**, **copy**), the atomic unit of work.

Run: ansible-playbook -i inv playbook.yml (-i = inventory file). Handy flags:

- check**, dry run: report what *would* change, change nothing
- list-tasks**, list the tasks without running them (preview)
- tags <t>** / **-skip-tags <t>**, run only / skip tasks tagged <t>
- ask-vault-pass**, prompt for the Vault password (decrypt vaulted vars)

```
- name: Example Playbook
hosts: web
become: true # sudo
gather_facts: false # skip setup (faster)
force_handlers: true # run handlers even if a
task fails

vars:
  packages: [nginx, curl]
  secret: "{{ vault_password }}"
roles:
  - myrole
tasks:
  - name: Install packages # list-as-option >
    loop (faster, deps OK)
    ansible.builtin.apt: # FQCN: namespace.
      collection: module
      name: "{{ packages }}" # alternative for
      apt is yum
      state: present
  notify: restart nginx
- name: Render config
  ansible.builtin.template: { src: app.conf.
    j2, dest: /etc/app.conf }
  when: enable_service
  tags: config
handlers:
  - name: restart nginx
    ansible.builtin.service: { name: nginx,
      state: restarted }
```

Loop anti-pattern: a per-item loop runs the module once per package – slower & can break on interdependencies; prefer the whole list in one name: (single transaction, as above):

```
- name: suboptimal # one apt/yum call per item
ansible.builtin.apt: { name: "{{ item }}",
  state: present }
loop: "{{ packages }}"
```

4.3 Ad-hoc, Modules & Errors

Ad-hoc (one module, no playbook): ansible -i inv -m <module> -a '<args>' <target>. The setup module gathers *facts* (what "Gathering Facts" runs before tasks). command vs shell: shell allows pipes/redirects/vars; *neither is idempotent*. ansible-inventory -i inv --list verifies the inventory; --check is a dry run. A playbook stops at the first failed task unless ignore_errors: true.

```
- name: validate
ansible.builtin.command: httpd -t
register: out
changed_when: out.failed
failed_when: false
- ansible.builtin.file: { path: /data, state:
  directory } # file/link/touch/absent
- ansible.builtin.lineinfile: { path: /etc/x.
  conf, line: "Include sites/*" }
```

4.4 Vault, Collections, Roles & Tags

Vault: ansible-vault create|edit|view|encrypt foo.yml; encrypt_string 'secret'

--name 'pw' inlines a single var. Pass via --vault-id @prompt|/path/file|name@script. Ciphertext safe to commit. **Collections** are bundles of modules/roles/plugins. FQCN ns.coll.mod (e.g. cisco.ios.ios_banner); independent versioning. Install: ansible-galaxy collection install <n> or via requirements.yml (collections: + roles:) then install -r. Sources: **Galaxy** (community), **Automation Hub** (RedHat, certified). **Roles:** ansible-galaxy init <n> creates dirs tasks/ handlers/ defaults/ vars/ files/ templates/ meta/ tests/; use via roles: list.

Tags can be used to execute a subset of tasks instead of the whole playbook. Run only specific tags by appending --tags <name> at the end of the ansible-playbook command. There are also two special commands: Tag *always* runs every time, except when explicitly skipped: --skip-tags=always. Tag *never* does not run unless specified with --tags=never

4.5 Jinja2 Templating

Rendered on **controller**, only result sent to target. {{ inventory_hostname }} = current target. Filters: ipaddr/ipv4/ipv6 (validate/version), hash('sha1'), default(x), min/max/shuffle.

```
{% for host in groups['webservers'] %}
BalancerMember http://{{ host }}:{{ port |
  default(80) }}
{% endfor %}
```

5 Kubernetes (K8)

Problems solved: automate deployment, scaling & management of containerized apps across many nodes: doing it by hand (placement, restarts, updates, scaling) doesn't scale.

Object: persistent record-of-intent. **Controller:** reconciles current → desired state (the *control loop*) → fault-tolerance (auto-replace failed pods), replica count (ReplicaSet), rolling updates + rollback (Deployment), autoscaling (HPA).

5.1 Architecture

Control plane: kube-apiserver (REST front), etcd (KV store), kube-scheduler (pod → node), kube-controller-manager (built-in controllers), cloud-controller-manager (cloud glue, optional). **Node:** kubelet (runs pods via container runtime), kube-proxy (Service networking/iptables).

5.2 Objects

Pod: smallest unit, 1+ containers share net/storage. **Sidecar:** container running alongside main in same pod (logs, proxy). **Init container:** runs to completion before main. **ReplicaSet:** keeps N pod replicas (rarely used directly). **Deployment:** manages ReplicaSets, rolling updates, rollback. **StatefulSet:** ordered pods with stable identity + per-pod PVC (DBs, brokers). **DaemonSet:** one pod per (matching) node: logs, CNI, monitoring. **Service:** stable VIP/DNS + LB over pod set. **Ingress:** L7 HTTP(S) endpoint. **Volume:** dir mounted into pod. **ConfigMap:** non-secret KV config. **Secret:** sensitive KV, base64-encoded, *not* encrypted by default. **HPA:** autoscales replicas on metrics. **CRD (Custom Resource Definition):** registers a new object *kind* to extend the K8s API; an **Operator** = custom controller that reconciles such custom resources (e.g. ServiceMonitor, Argo Application, CiliumNetworkPolicy).

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: app }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 1, maxSurge: 1 }
  template:
    metadata: { labels: { app: web } }
    spec:
      initContainers:
        - { name: init, image: busybox, command: ['sh', '-c', 'sleep 10'] }
      containers:
        - name: web
          image: nginx
          ports: [{ containerPort: 80 }]
        - { name: sidecar, image: busybox, command: ['sh', '-c', 'while true; do sleep 30; done'] }
```

5.3 Services

Give stable virtual IP + DNS over label-selected pods, round-robin LB. **ClusterIP** (default) internal only; **NodePort** static port on every node <NodeIP>:<port>; **LoadBalancer** cloud LB w/ public IP; **ExternalName** CNAME alias to external DNS (no proxy). **Ingress** routes HTTP(S) by host/path (needs controller: Nginx, Traefik). **Gateway API** (Gateway+HTTPRoute/TLSRoute) – modern, multi-protocol, role-oriented Ingress replacement. *Port trap:* Service port vs targetPort (container) vs nodePort; Ingress backend refs Service port.

5.4 ConfigMaps & Secrets

Inject into Pods as a single env var (configMapKeyRef/secretKeyRef),

all keys at once (envFrom), or mounted as files (volume).

```
env:
- name: APIKEY
  valueFrom: { configMapKeyRef: { name: cfg, key: api-key } }
envFrom:
- configMapRef: { name: cfg }
volumeMounts:
- { name: v, mountPath: /etc/cfg }
volumes:
- { name: v, configMap: { name: cfg } } # or:
  secret: { secretName: s }
```

5.5 Storage: PV, PVC, StorageClass

Container FS is ephemeral. **Volume:** dir in pod, lives w/ pod (emptyDir, hostPath, NFS, ...). **PersistentVolume (PV):** cluster-scoped piece of external storage. **PersistentVolumeClaim (PVC):** namespaced request for storage (size, accessMode); binds to a matching PV. **StorageClass:** dynamic provisioner – creates a PV on-demand if no static PV matches (e.g. gp3, standard-rwo).

5.6 LimitRange & ResourceQuota

LimitRange: per-container min/max + default req/lim in NS; out-of-range Pod rejected, missing req/lim auto-injected. **ResourceQuota:** caps summed NS usage; over-quota Pod rejected.

5.7 Namespaces

Logical isolation in one cluster (RBAC, quotas). Namespaced (Pods, Services) isolated; cluster-wide (Nodes, PV) shared. Defaults: default, kube-system, kube-public, kube-node-lease.

5.8 Rolling Updates & Rollback

Deployment ensures availability during change. **Recreate:** kill all → recreate (downtime). **RollingUpdate** (default): tuned by maxUnavailable (max pods down) + maxSurge (max extra pods). **History & rollback:** Deployment records a revision history, `kubectl rollout history deploy/x` lists revisions, `rollout undo deploy/x --to-revision=N` reverts. **Reliable config over time:** imperative is fast but hard to track/maintain → keep object YAML in **Git** (version control); alternative rollback = restore the file from an earlier commit & `kubectl apply`.

5.9 Scheduling

kube-scheduler picks node via **Filtering** then **Scoring**. **Filtering** (feasibility): PodFitsHostPorts, PodFitsResources, PodMatchNodeSelector, CheckNodeDiskPressure, CheckVolumeBinding. **Scoring** (rank): SelectorSpreadPriority (spread same Service/STS/RS), LeastRequestedPriority (free resources), NodeAffinityPriority.

```
spec:
  nodeSelector: { disktype: ssd }
```

Taints on nodes repel pods; **Tolerations** on pods allow (don't force) them onto tainted nodes – pair w/ nodeSelector/affinity to pin. Effects: *NoSchedule*, *PreferNoSchedule*, *NoExecute* (evict). No effect on DaemonSets. CLI: `kubectl taint nodes <n> gpu=true:NoSchedule`, `kubectl label nodes <n> disktype=ssd` (trailing - removes).

```
tolerations:
- { key: "gpu", operator: "Equal", value: "true", effect: "NoSchedule" }
```

cordon/drain: `kubectl cordon <n>` marks unschedulable (existing stay); `drain <n>` also evicts pods (maintenance); `uncordon <n>` reverts.

5.10 Probes & Resources

kubelet probes via `httpGet/tcpSocket/exec`. **Liveness:** fail → restart. **Readiness:** fail → remove from Service endpoints. **Startup:** guards slow boots, disables others until OK. Repeated liveness fail → exp backoff → CrashLoopBackOff. **requests:** reserved min (scheduling). **limits:** ceiling: CPU throttled, memory over-limit → OOMKilled.

```
resources:
  requests: { cpu: "250m", memory: "64Mi" }
  limits: { cpu: "500m", memory: "128Mi" }
livenessProbe:
  httpGet: { path: /healthz, port: 80 }
  initialDelaySeconds: 5
  periodSeconds: 10
readinessProbe:
  httpGet: { path: /ready, port: 80 }
```

5.11 Autoscaling (HPA)

HorizontalPodAutoscaler: auto-scales a workload's **replica count** on observed metrics, holds performance during traffic spikes, cuts cost at low usage. **Key config:** `minReplicas`, `maxReplicas`, `target` metric (threshold that triggers scaling, e.g. 70% CPU). **Metrics:** *resource* (CPU/memory util, needs **Metrics Server**), *custom* (app-specific via adapters), *external* (traffic-based, e.g. requests/s). **Best practice:** pods must set resource `requests/limits`, HPA computes % against them (none → no scaling); test scaling under load before prod.

```
kubectl autoscale deploy php-apache --cpu-percent=50 --min=1 --max=10
```

5.12 Imperative vs Declarative

3 ways to manage objects: **Imperative commands:** act on live objects straight from the CLI (`kubectl run/create/expose/scale/delete`); fast for one-off/learning, but not reproducible, no single source of truth, hard to review.

Imperative object config:

file + explicit verb (`kubectl create|replace|delete -f f.yaml`); replace *overwrites* the object (drops changes not in the file). **Declarative config:** describe desired state in YAML, K8s computes & applies the diff (`kubectl apply -f <dir/>`, even a whole folder); idempotent, repeatable, Git-friendly, `kubectl diff -f` previews. *Recommended* (course: more control than CLI-only). **Workflow:** scaffold with imperative `--dry-run=client -o yaml`, then manage declaratively via `apply`.

5.13 Commands

Apply: `kubectl {create|apply|replace} -f manifest.yaml`
Gen YAML: `kubectl create deploy web --image=nginx --dry-run=client -o yaml > d.yaml`
Imperative: `kubectl run, scale deploy x --replicas=4, expose`
Update image: `kubectl set image deploy/x web=nginx:1.27`
Inspect: `kubectl get pods -A -o wide`, describe node `<n>`, get nodes `--show-labels`
Exec: `kubectl exec -it nginx-xxx -- sh (drop -it for one-shot)`
Rollout: `kubectl rollout {status|history} deploy x, rollout undo deploy x --to-revision=N`
Node ops: `kubectl cordon|uncordon|drain <n>`
Default ns: `kubectl config set-context --current --namespace=lab`

6 Helm

Package manager for K8 (apt/brew-like). **Chart:** files describing related K8 resources. **Release:** running instance. **Repo:** stores versioned charts. **Architecture:** client-side only (Helm 3, no Tiller); talks to K8 API directly; release state stored in-cluster as **Secrets** (in the release's namespace). **Pros:** huge chart library, versioning/deps, lifecycle (install/upgrade/rollback), powerful Go-template logic. **Cons:** templates hard to debug, "spaghetti YAML", opaque state, steep curve. Use for: distributing complex apps, consuming community charts, advanced logic. Avoid for: simple YAML tweaks.

6.1 Commands

```
helm repo add|list|rm NAME URL # repos
helm search repo <> / <> -l # all versions
helm install -f vals.yaml NAME CHART --version 1.2.3
helm upgrade RELEASE CHART
helm rollback RELEASE REVISION
helm uninstall RELEASE_NAME # also drops history
helm list; helm history RELEASE_NAME
```

```
helm status RELEASE_NAME # unknown|deployed|uninstalled|
# superseded|failed|uninstalling|pending-{install,upgrade,rollback}
helm get all|values|notes|manifest
RELEASE_NAME # values=overrides
helm template NAME CHART -f v.yaml --set k=v # render local
helm install NAME CHART --dry-run --debug # simulate
helm lint CHART_PATH [--with-subcharts]
helm show chart|readme|values|all CHART
helm package CHART_PATH # build .tgz, then helm push <pkg> <repo>
helm dependency update CHART # pulls into charts/
```

6.2 Chart Structure

`helm create NAME` scaffolds: **Chart.yaml** metadata, **values.yaml** default values, **templates/** Go-template manifests, **templates/_helpers.tpl** named templates (≠not rendered), **templates/NOTES.txt** rendered & shown post-install, **charts/** subcharts; **crds/** CRDs, **.helmignore** ignore globs (!=negate)

6.3 Chart.yaml fields:

apiVersion v2=Helm 3+, v1=legacy
name chart name;
version chart version, SemVer2 (e.g. 1.4.0) → name-version.tgz
type application (default) | library (helpers only, no install)
appVersion app shipped (info-only)
kubeVersion allowed range (e.g. >=1.12 <1.14); install fails if mismatch
dependencies name/version/repository/condition/alias (requirements.yaml deprecated; lockfile on update)

6.4 Templating

Objects: `.Release`, `{}.Name`, `{}.Namespace`, `{}.Revision`, `{}.IsInstall`, `{}.IsUpgrade`, `{}.Chart.*`, `{}.Values`, `{}.Files.Get`, `{}.Capabilities`, `{}.Template`.
Values precedence (low→high, last wins): subchart `values.yaml` → parent `values.yaml` → `-f my.yaml` → `--set k=v`; unset via `--set x=null`.
Functions (Sprig, ~60+): **upper**, **quote**, **default**, **trunc**, **toYaml**, **indent**/**nindent**. | pipes result as last arg. **include** = pipeable **template**. **required** fails if empty. **tpl** renders a string as a template.
Flow: `{{if}}...{{end}}` (false on 0//nil/empty); `{{range $k,$v := .list}}`; `{{with .sub}}` rescopes. **Whitespace:** `{{- trims left, -}}` trims right; `nindent N` = newline+indent. **YAML indent** is critical.

```
{{- define "app.name" -}}{{ default .Chart.Name
.Values.name | trunc 63 }}{{- end }}
name: {{ include "app.name" . | quote }}
{{- required "set .Values.img!" .Values.img }}
resources:
- {{ toYaml .Values.resources | nindent 4 }}
```

7 Kustomize

Template-free overlay tool, built into `kubectl (-k)`. `kustomization.yaml` lists **resources** + declarative **transformers** (cross-cutting fields) + **patches** (partial spec edits) + **generators**. **Pros:** pure YAML, native to `kubectl`, easy diff/review, easy to learn. **Cons:** less powerful than Helm, no deps mgmt, hard to package/distribute, limited conditionals. **Use for:** env-specific configs (dev/prod), avoiding templating. Often combined with Helm.

7.1 Commands

```
kustomize create --resources deploy.yaml,svc.yaml # scaffold
kustomize edit add resource|set image|add label|remove resource
kustomize build <dir> # render to stdout
kustomize build <dir> | kubectl apply -f -
kubectl apply -k <dir> # native
kubectl kustomize <dir> # render only (no apply)
kubectl delete -k <dir>
kustomize cfg fmt <dir> # format yaml
kustomize localize <dir> # bundle remote URLs/git locally
kustomize edit fix # auto-migrate deprecated syntax
```

7.2 kustomize.yaml

Deprecated → new: `commonLabels` → `labels`, `patchesStrategicMerge/patchesJson6902` → `patches`, `bases` → `inside resources`:
`apiVersion: kustomize.config.k8s.io/v1beta1`
`kind: Kustomization`
`resources: [.././base, storage.yaml] # files, dirs, urls`
`namespace: prod # transformers`
`namePrefix: dev-`
`nameSuffix: -v1`
`commonAnnotations: {version: 1.0.0}`
`labels: # replaces commonLabels`
`- includeSelectors: true`
`pairs: {app.kubernetes.io/name: hello}`
`images: # change tag/name w/o editing Deployment`
`- {name: nginx, newName: bitnami/nginx, newTag: latest}`
`patches: # strategic or JSON6902`
`- target: {kind: Service, name: nginx-app} patch: |-`
`- {op: replace, path: /metadata/name, value: nginx-server}`
`configMapGenerator: # hash suffix appended -> change forces rolling restart`
`- name: app-config literals: [LOG_LEVEL=debug] files: [config.env] # or files:`
`secretGenerator: # base64-encodes; same hash mechanism`
`- {name: ex-secret, literals: [user=admin, pass=secret]}`
`components: [.././components/mysql]`

ConfigMap/Secret modes: (1) *Traditional:* declare as resource → change does **not** restart pods (silent drift). (2) *Generator:* `configMapGenerator/secretGenerator` appends content hash to name → references rewritten → pods rolling-restart on change.

7.3 Bases, Overlays, Components

Base: common (base/ with kustomization.yaml). **Overlay:** env-specific dir (overlays/dev, overlays/prod) that lists base in resources: + adds patches/transformers → produces a **variant**. **Component:** kind: Component (v1alpha1); reusable mix-in chunk (resources+patches+generators), included via components: in overlays. resources = full manifests/bases; bases: key itself is deprecated.

```
# components/mysql/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1alpha1
kind: Component
resources: [deployment.yaml, service.yaml]
```

7.4 Best Practices & Pitfalls

Keep common values (namespace, labels) in base. Organize by kind (services.yaml, hpa.yaml). Standard dirs: base/, overlays/, components/, patches/. **Pitfalls:** nested overlays (flatten!); not pinning image tags (use images: with explicit newTag); drift between cluster + kustomization (no release state tracked, unlike Helm); skipping kustomize build validation before apply; using traditional ConfigMaps when rollout-on-change needed (use generator).

7.5 Helm vs. Kustomize

Helm: templating + package manager (variables via values.yaml, releases, repos, rollbacks); good for distributing reusable apps. **Kustomize:** template-free overlays (patch/merge plain YAML per environment), built into kubectl (-k); good for own manifests across dev/test/prod. The two are often combined (Helm to package, Kustomize to patch).

8 K8s Monitoring & Logging

Observability is a property (degree system generates actionable insights, qualitative); **Observing/Monitoring** is a process (collect/store/evaluate signals). **Three pillars:** **Metrics** (quantitative: util, counters, rates), **Logs** (qualitative: stack traces, events), **Traces** (request path across services via spans - latency/bottlenecks, e.g. Tempo, Jaeger follow one request across services via spans to find latency/bottlenecks, e.g. Tempo, Jaeger). **Monitoring vs Logging** complement each other.

K8s obs gaps: no historical metrics, kubectl logs only running containers (lost when container dies, kubelet cleanup), no native dashboards/alerts. Container logs at /var/log/pods/...;

kubelet rotates at 10MB, keeps 5. **Push (agent-based):** agent on system sends to collector; observed decides when. **Pull (agentless):** collector queries via SNMP/ICMP/HTTP; collector decides when, but system must be online. **Golden Signals** (Google SRE): latency, traffic, errors, saturation.

PLG stack (lightweight, K8s-native): Prometheus+Loki+Grafana. **LGTm:** Loki, Grafana, Tempo, Mimir. **EFK/ELK:** Elasticsearch (Java, JSON index, full-text, heavy) + Kibana + Fluentd/Logstash. PLG indexes labels only (cheap, fast, weaker text search). **OpenTelemetry:** unified telemetry (metrics/logs/traces) spec.

8.1 Prometheus & PromQL

Go, TSDB, pull-based scrape over HTTP (/metrics). Pushgateway/remote-write for short jobs. Client libs Go/Java/Python/Ruby/Rust. Standalone (no remote dep). **Targets:** Kubelets, kube-state-metrics, exporters. **Exporters:** bridge for apps without native /metrics (PostgreSQL, NGINX, Redis, JIRA, Kafka, ...); use when can't instrument. **node-exporter:** HW/OS. **kube-state-metrics:** K8s object state. Install metrics-server for kubectl top. CPU units: 100m=0.1 CPU absolute. **Metric types:** Counter (monotonic, reset to 0: requests/errors), Gauge (current value: memory, concurrency), Histogram (sampled buckets), Summary (sampled + total/sum). **Format:** name{label=val,...} value [ts], foundation of **OpenMetrics** std. **PromQL data types:** Instant vector (set of series, one sample each, same ts), Range vector (range per series, e.g. [5m]), Scalar (float), String (unused). **Offset:** (positive = backwards): rate(x[5m] offset 1w). **@-modifier:** sum(x 1609746000). histogram_quantile(0.95, rate(..._bucket[5m])) for p95 from Histogram. rate() useless on spikes/outliers - use raw & > threshold.

```
sum by (namespace) (kube_pod_status_ready{
  condition="false"})
rate(mysql_global_status_commands_total{command
=~"select"}[5m])*100 > 35
```

Prometheus Operator (installed via the kube-prometheus-stack Helm chart): deploys operator + Prometheus + Grafana + Alertmanager in one go, and lets you configure monitoring

declaratively through CRDs instead of editing prometheus.yml.

CRDs: Prometheus (a Prometheus instance), ServiceMonitor/PodMonitor (declare which Services/Pods to scrape).

How a target gets scraped - a chain of label selectors, each picking the next level down: Prometheus.serviceMonitorSelector → **ServiceMonitors**, each ServiceMonitor's selector → **Services**, the Service's own selector → **Pods**. Drop in a ServiceMonitor and its pods are auto-scraped - no Prometheus config edit.

Scaling Prometheus: Federation (hierarchical: region scrapes DC); Thanos (sidecar, global query, long-term, dedup); Grafana Mimir (horiz. scalable, ingester/querier components, multi-tenant, remote-write only).

8.2 Grafana, Alerting

General-purpose viz, many data sources (Prom/Loki/ES). Explore feature: browse data sources, build queries, debug, share. OAuth/SAML/LDAP. Dashboard JSON repo grafana.com/grafana/dashboards. Tips: important data top, status-light > number, simplicity, Data Source variable.

Alertmanager vs Grafana Alerts: Alertmanager (Prom ecosystem, GitOps-friendly, separates viz/alert, Prom-only). Grafana Alerts (any data source, tied to dashboards, can chain to ext Alertmanager, less GitOps-friendly). Flow: Rule → evaluate/threshold → Alert → Silence? → AlertManager → Channels (Email/Slack/Teams/Discord/Threema/Webhook) → Contact Points. **Warning ≠ Alert:** cert expires in 45d, CPU 75%, budget 200% are warnings not pages. Avoid alert fatigue, build up slow, test.

8.3 Logging: Loki, Alloy

Architectures: Node-level (DaemonSet agent reads node log file - survives container death, batched); App-level (app pushes to backend directly); Sidecar (streaming = sidecar writes to file then node agent ships - kubectl logs won't show sidecar's own; logging-agent sidecar = ships to backend directly, low-latency but loses on disconnect, adds complexity, misses crashes). Best practice: structured JSON logs.

Loki: horiz. scalable, indexes labels only (not text), nanosecond unix ts, cheap storage, PromQL-like **LogQL:** {job=ttest"} |!= error"; metric queries rate({app=*"} |="176"[1m]); filters |!= contains, |!= regex, |~. **Alloy** (Grafana, replaces Prom-

tail): collects all telemetry, own HCL-like lang. Components: loki.source.file (tail files, classic DaemonSet pattern), loki.source.kubernetes (subscribe via K8s API - no root, no DaemonSet needed, but more CPU/net), loki.source.kubernetes_events (K8s events). Logs enriched with **K8s metadata**, forwarded over HTTP. Helm-installable.

Pitfalls: too much logging → storage/complexity; no alert tuning → noise/missed; forgetting rotation/archival; relying on single signal; logging secrets.

9 Service Mesh

9.1 Microservices

Small independent services, one business function, communicate via lightweight APIs. Microservices == distributed computing.

Pros: independent deploy, flexibility, scalability, fault isolation, team autonomy.

Cons: operational complexity, distributed-system challenges, data consistency, higher resource use.

Challenges: Communication: services must know endpoints, new service ripples changes. Security: in-cluster traffic unsecured, any pod → any pod. Monitoring: spread out, logic must be added per app.

9.2 Distributed Computing Fallacies

Network reliable; latency zero; bandwidth infinite; topology stable; one admin; transport cost zero; network homogeneous. All false → motivate mesh.

9.3 Service Mesh

Dedicated infra layer handling service-to-service communication: traffic routing, security, observability, resiliency - abstracted from app code.

Capabilities: Traffic Mgmt (routing, splitting), Resilience (fault tolerance, retries, fallback), Security (mTLS, auth-N/Z), Observability (metrics, logs, tracing).

Why: centralized policy, uniform observability, faster incident resolution, transparent security, advanced rollouts (canary/AB), resilience.

Components: Data Plane: lightweight proxies (Envoy), handle comm/metrics/security. Control Plane: manages config/policies (Istiod), pushes rules to data plane.

9.4 Mesh Evolution: Library → Sidecar → Kernel

Library: mesh logic linked into each app, per-language lib (Python, Go, ...). Invasive, language-locked.

Sidecar: proxy per pod, transparent traffic interception, no code change, language-agnostic. Downsides: invasi-

ve pod-spec mutation, upgrade = pod restart, large CPU/RAM per pod (worst-case provisioning), extra hop.

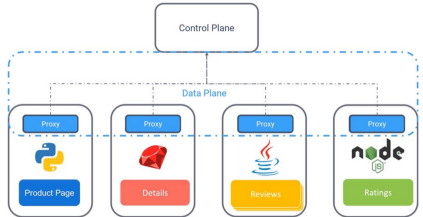
Kernel/Ambient: mesh in kernel (eBPF) or per-node agent. 30 pods/node → 1 proxy/node instead of 30. L7 still typically needs userspace proxy.

Leading meshes: Istio (Google, feature-rich), Linkerd (lightweight, perf), Kuma (CNCF, VMs+containers), Consul Connect (HashiCorp, discovery), Cilium (eBPF, not dedicated mesh).

9.5 Sidecar Pattern Pros/Cons

Pros: + transparent (no code change, lang-agnostic), + per-pod isolation, + platform team owns cross-cutting (mTLS/retries/routing/telemetry).

Cons: - CPU/RAM scales with pod count, - extra hop = latency, - sidecar injection + proxy upgrade needs restart, - overkill for few services.



Envoy proxies authenticate via mTLS (mutual TLS, both sides present certs).

9.6 Ambient Mesh (Sidecarless Istio)

Sidecar downsides → ambient: per-node zTunnel agent (L4, mTLS, auth-N/Z, no HTTP parsing) communicates across nodes via HTTPS CONNECT tunnel. Optional L7 Waypoint Proxy per namespace (Envoy) for rich policies/telemetry. zTunnel alone = L4 metrics only.

Slicing: Secure Overlay (L4): TCP routing, mTLS tunneling, simple authZ, TCP metrics. **L7 Processing:** HTTP routing/LB, circuit breaking, rate limiting, fault injection, retry, timeouts, rich authZ, HTTP metrics/access logs/tracing. **Cilium variant:** dynamic eBPF programs in kernel handle L3/L4 (network path); per-node Envoy proxy for L7 termination when needed. No sidecar, no extra hop for L4.

9.7 Gateway API & Ambient Routing / GAMMA

Enable ambient: kubectl label ns x istio.io/dataplane-mode=ambient (sidecar mode: istio-injection=enabled). Routes via K8s Gateway API (HTTPRoute) not VirtualService. Two Deployments behind one Service ≈50/50 default; override via weights. Best practice: version label per Deployment.

GAMMA (Gateway API for Mesh Management & Administration): Gateway API for east/west (mesh) traffic, not

just north/south (ingress). Adopted by more meshes.

```
# weighted canary 90/10:
- backendRefs:
  - { name: frontend, port: 5000, weight: 90 }
  - { name: frontend-v2, port: 5000, weight: 10 }
}

# mirror/shadow (copy to v2, drop response):
- backendRefs:
  - { name: frontend, port: 5000 }
  filters:
    - type: RequestMirror
      requestMirror: { backendRef: { name:
        frontend-v2, port: 5000 } }
```

istio-system pods: Istiod, Ingress Gateway (Envoy), Prometheus, Grafana, Jaeger, Kiali. **Kiali shapes:** rectangle=app, triangle=Service, square=workload/pod.

9.8 Traffic Management (Istio Classic)

Gateway replaces K8s ingress/egress. VirtualService binds gateway, routes to services. DestinationRule applies post-routing policy: LB, connection pool, outlier detection (circuit breaking), TLS mode, defines subsets (versions).

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata: { name: my-gateway }
spec:
  selector: { istio: ingressgateway }
  servers:
    - port: { number: 80, name: http, protocol:
      HTTP }
      hosts: ["example.com"]
  --
kind: VirtualService
metadata: { name: my-vs }
spec:
  hosts: ["example.ch"] # must match gateway
  host
  gateways: [my-gateway]
  http:
    - route:
      - destination:
        host: my-service
        weight: 99 # split traffic if 2nd dest
        defined
      port: { number: 80 }
  --
kind: DestinationRule
metadata: { name: my-dr }
```

```
spec:
  host: my-service
  trafficPolicy:
    tls: { mode: ISTIO_MUTUAL }
```

9.9 Resilience & Use Cases

Fault Injection: simulate net failures, test error handling. **Timeouts:** cap request time, avoid queue buildup. **Retries:** auto-retry on failure. **Cascading Failure:** one service dies → callers pile up → whole graph red. **Circuit Breaking:** trip on errors/outlier → prevent cascading failure (DestinationRule outlierDetection). **Rate Limiting:** cap RPS, protect downstream from overload. **Zero-Trust:** encrypt + authenticate every call, no implicit trust in cluster. Istio exports Prometheus metrics by default.

9.10 Deployment Strategies

Rolling: gradual pod replace, default K8s. **Recreate / Big-bang:** kill all, deploy new (downtime). **Canary:** small % to v2, watch, ramp up. **Blue/Green:** two envs, switch traffic atomically, easy rollback. **Dark Launching / Shadow:** mirror prod traffic to v2, drop response (test under real load). See [General Definitions](#).

9.11 Security

AuthN: mTLS between services. **AuthZ:** AuthorizationPolicy = fine-grained RBAC.

```
# enforce mTLS in namespace
kind: PeerAuthentication
metadata: { name: default, namespace:
  production }
spec:
  --
  mtls: { mode: STRICT }
  --
kind: AuthorizationPolicy
metadata: { name: require-jwt, namespace: foo }
spec:
  selector: { matchLabels: { app: httpbin } }
```

```
action: ALLOW
rules:
- from: [{ source: { namespaces: ["frontend"]
  } }]
  to: [{ operation: { methods: ["GET"], paths:
    ["/headers"] } } ] }
```

10 Cilium & eBPF

10.1 eBPF

eBPF (extended Berkeley Packet Filter): runs sandboxed programs directly in the Linux kernel without changing kernel source or loading modules. Programs attach to hooks (syscalls, network events) and are verified before loading. Enables high-performance networking, observability and security at kernel level.

10.2 Cilium

A CNI (Container Network Interface) plugin built on eBPF. Replaces kube-proxy: instead of one iptables rule chain per Service, it programs eBPF maps in the kernel (faster, scales better). **Identity-based security:** Cilium derives a security *identity* from a pod's labels (not from its IP). Policy is enforced on identities, so it survives pod rescheduling and IP changes.

Hubble: Cilium's observability layer; gives network flow visibility (service map, L3–L7 flows).

10.3 Network Policies

Declarative firewall rules for pod traffic. Plain NetworkPolicy works at L3/L4 (IP, port, protocol). CiliumNetworkPolicy adds L7 (HTTP method/path, DNS, gRPC, Kafka).

Default deny: as soon as a pod is selected by any policy, all non-matching traffic is dropped (whitelist model). Rules split into **ingress** (incoming) and **egress** (outgoing) and are *stateful*:

return traffic of an allowed connection is permitted automatically.

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: allow-frontend
spec:
  endpointSelector:
    matchLabels:
      app: backend # applies to backend pods
  ingress:
    - fromEndpoints:
      - matchLabels:
        app: frontend # only frontend may connect
      toPorts:
        - ports:
          - port: "80"
            protocol: TCP
          rules:
            http: # L7 rule
              - method: GET
                path: "/api/*"
```

11 GitOps

Git = single source of truth for infra/app config; in-cluster agent reconciles live → desired state. **4 Principles (OpenGitOps):** **declarative** state; **versioned & immutable** (history = audit, revert = rollback); **pulled** by agents; **continuously reconciled** (detect + self-heal drift).

11.1 Push vs. Pull

Push (classic CI/CD): pipeline holds creds, runs kubectl apply. + fast, well-known; - creds outside cluster, inbound firewall, no drift detection, per-env pipeline edits. **Pull (GitOps):** in-cluster agent watches Git, syncs. + no inbound access, drift correction, multi-cluster scale; - agent per cluster, polling latency. **Trust:** CI server → agent + Git. **Access dir:** push = in; pull = out.

11.2 Argo CD vs. Flux CD

Argo CD: Web UI + CLI, multi-cluster from one UI, SSO/RBAC, sync waves, pre/post hooks, health checks. CRDs:

Application (repo+path → cluster/ns + syncPolicy), ApplicationSet (templated per branch/cluster/env), AppProject (restrict what/where/who). Pattern: **App-of-Apps** (root App syncs children).

Flux CD: CLI-first, no UI, modular **GitOps Toolkit** (source, kustomize, helm, notification, image). Supports Gitless GitOps (OCI/S3) + image automation. **Naming clash:** kustomize.config.k8s.io (feature) vs. kustomize.toolkit.fluxcd.io (Flux CRD).

11.3 Secrets & Best Practices

Encrypted in Git: Sealed Secrets (controller decrypts), SOPS. **Referenced:** External Secrets, Secrets Store CSI driver (mounts Vault/KMS as volume).

Monorepo: clusters/<c>/bootstrap, apps/base/ shared, apps/<c>/ per-cluster. **Rules:** never commit plaintext, pin image tags (no latest), monitor controller, split monolith repos.

12 Site Reliability Engineering

"Keep the site up, whatever it takes" **SL Indicators:** A measurement (e.g. Response latency over a period of 10 min) **SL Objectives:** A Objective (e.g. 99.9% below 5ms) **SL Agreements:** A economic incentive (e.g. or less bonus) **Error Budget:** 1 - SLO (e.g. 100% - 99.9% = 0.1%)

The four golden signals: latency, traffic, errors, and saturation.

12.1 Outage Handling

Goals: 1. Minimize impact (Automated, fast detection; good diagnostic information, practice remediation) 2. Prevent reoccurrence (handle event, write post-mortem)